Aerial Robotics Kharagpur Selection Task 2

Harshil Mishra

VISION-BASED LINE FOLLOWER

***Abstract*—Line Follower Drone**

The goal was to implement a Line Follower algorithm using camera feed.

I. INTRODUCTION

Autonomous navigation enables drones to perform tasks without human control by using onboard sensors and vision systems. In this project, a vision-based line follower algorithm is developed using the drone's downward-facing camera. The system processes the camera feed to detect a ground track line and calculate its angle deviation from the center. This angle is sent to the flight controller to adjust the drone's direction and keep it aligned with the path. The solution is implemented by modifying the Image Processing System in the Simulink model.

II. PROBLEM STATEMENT

The objective of this project is to develop a vision-based line following algorithm for a drone using the downward-facing camera feed. The system must process the captured images to detect a ground track line and determine its orientation relative to the drone's center. The algorithm should calculate the line angle (deviation from the center) and send it to the flight controller for steering adjustments. Implementation is limited to modifying the Image Processing System in the Simulink model. The performance will be evaluated based on the accuracy of the angle, and the run fails if the drone loses the line or crashes.

III. RELATED WORK

1. MATLAB (MATLAB R2024b): MATLAB is used to implement the line follower algorithm and process the image data obtained from the drone's camera. In this project, MATLAB functions are used to perform operations such as pixel analysis, centroid calculation, and steering control logic.

2. Simulink: Simulink provides a graphical environment for designing and simulating control systems. The provided model parrotMinidroneCompetition.slx contains the drone control architecture, where the Image Processing System block is modified to implement the line detection and navigation algorithm.

3. Parrot Minidrone Platform: The Parrot Minidrone is the UAV platform used in this project. It includes a downward-facing camera, sensors, and wireless communication that allow it to capture images and execute control commands generated from the Simulink model.

4. Simulink Support Package for Parrot Minidrones: This package enables integration between MATLAB/Simulink and the drone hardware. It allows the system to access the drone's camera feed, send control signals, and retrieve telemetry data during simulation and real flight.

5. Color-Based Line Detection The algorithm detects the track line using color thresholding. By comparing the intensity values of the RGB channels, red pixels belonging to the track line are separated from the background floor to create a binary mask of the line.

6. Centroid-Based Line Position Estimation After identifying the red pixels of the track, the centroid (average position) of these pixels is calculated. This centroid helps determine the lateral deviation of the line from the center of the image frame.

7. Proportional Control for Steering A proportional controller (Kp) is used to convert the position error into a steering command. The error between the centroid and the image center determines the **Y-axis steering value**, allowing the drone to adjust its direction smoothly.

8. Adaptive Speed Control The drone's forward velocity (X-axis) is dynamically adjusted depending on the steering intensity. When the drone needs to turn sharply, the speed is reduced to maintain stability and prevent losing the track.

9. Landing Zone Detection The algorithm also includes detection of a circular marker at the end of the track. By analyzing the width, height, and density of the detected red region, the system identifies the landing circle and triggers a stop command for the drone.

IV. INITIAL ATTEMPTS

My initial approach to the line-following problem was based on simple color thresholding and global centroid calculation.

1. Global Centroid and Lateral Oscillation: Initially, I calculated the center of the path by averaging the coordinates of all detected red pixels across the entire frame. The logic was to steer the drone so that this average center stayed at the horizontal midpoint of the camera view. This led to clean lateral oscillation. Because the drone was looking at the entire frame, a corner appearing at the top of the image would pull the centroid to the side earlier. This caused the drone to start turning while still on the straightaway, overshoot the line.

2. Early Intersection Triggering: As the drone approached a corner, the horizontal bar of the intersection would dominate the red pixel count. My initial code interpreted this as a command to turn immediately. For a 90-degree rectangle, the drone must remain unresponsive to the upcoming path until it is physically standing on the intersection. I realized that by not filtering the vertical field of view, I was allowing distant data to destroy the immediate steering.

4. Implementation of Skeletonization: To solve these issues, I transitioned from "blob tracking" to a row-by-row skeletonization method. Instead of one single point, I generated a

”spine” of midpoints for every horizontal line in the camera view to determine the path’s true geometry.

By selecting a specific look-ahead point on this skeleton, I created a steering vector. This separated the drone’s current position from its intended direction, providing the stability needed to navigate sharp corners and maintain a centered position on the return leg of the rectangle.

V. Final Approach

Color Extraction I isolate the red line by checking that the red channel is above 110 and at least 40 units higher than both green and blue. This reliably removes the Grey floor.

Lost Line Recovery If no red pixels are found, I output X = 0.3, Y = 0 to keep the drone creeping forward until the line comes back into view.

Circle Detection I check three things together — bounding box fill density above 0.65, width above 38 pixels, and no red pixels near the top of the frame. All three must be true simultaneously to trigger a stop (X = 0, Y = 0). This prevents false stops at corners, which also have dense bounding boxes but still have line extending ahead.

True Centerline Extraction For each row, I take the midpoint of the leftmost and rightmost red pixel instead of just finding red pixels. This gives the true center of the thick line and eliminated the constant left drift I was seeing.

Pure Pursuit Steering I scan skeleton points from the top of the frame downward and steer toward the first one found in the top half. This means the drone reacts to where the line is going, not where it currently is — fixing corner handling completely.

Unit Vector Output I draw a vector from drone position (row 120, column 40) to the target point and normalize it. X is forward, Y is lateral. Column 40 is used instead of 80 due to a physical camera offset discovered through testing.

Speed Scaling X is scaled by 0.45, Y by 1.0. This makes lateral response fast on turns while keeping forward motion conservative.

VI. Results and Observation

The drone successfully followed the red line across straight sections, diagonal paths, and corners of varying angles, stopping correctly upon detecting the circle at the end of the track.

Observations:

Circle Detection —¿ Stopped Early: My initial circle detection used width as the size check. The problem was that the drone stopped early because a wide corner also satisfied the width threshold. I fixed this by switching to density concept rather than width and area.

The lineEnded Logic Gap: When the drone approached the circle, it would start moving vertically before stopping. The root cause was the ‘lineEnded‘ flag — the code was waiting for red pixels to disappear from the top of the frame before triggering a stop. While waiting, the look-ahead target vanished from the top rows, the lateral error dropped to zero, and the drone straightened out and drifted vertically. The drone was essentially in a blind spot between following the line and seeing the circle.

Camera Offset (CENTER = 45): Through testing I found the drone had a consistent left drift. This turned out to be a physical camera misalignment. Shifting the reference column from 80 to 45 corrected the drift on straight sections without touching any steering logic.

VII. Future Work

1. A Better way to fix the offset: Currently, the offset is dealt by manually tuning and trying values to make the drone on the red line. But the value to set varies from path to path for example it is different for right slant, straight and left slant path. also it will depend on which height the drone is flying.

2. A accurate approach to take sharp turns: The drone does not perform as good at sharp turns with angle similar to 90 degrees. There should be a better approach for that.

3. Adjusting the Red Color: Adaptive color thresholding would improve robustness so red detection holds up under different lighting conditions rather than relying on fixed values.

4. Altitude-Dependence: Altitude-aware circle detection would make stopping more reliable. At higher altitudes the circle appears smaller in the frame, so the area threshold should scale with estimated drone height rather than being a fixed number.

Conclusion

In this project, a vision-based line follower algorithm was successfully implemented for the Parrot Minidrone using MATLAB R2024b and Simulink. The system processes the downward-facing camera feed to detect the track line and calculate the deviation angle for steering control. By using color-based line detection, centroid calculation, and proportional steering control, the drone is able to autonomously follow the path. The algorithm was tested in the simulation environment to ensure accurate angle estimation and stable navigation.

NOISE FILTERING

***Abstract*—Noise Filtering for Cleaning Images**

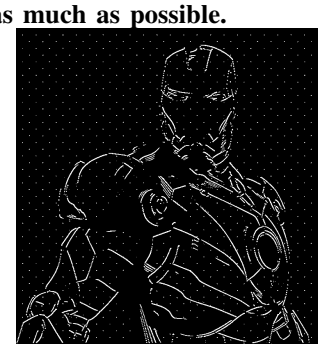
In this project, two corrupted images were cleaned using various noise filtering techniques.

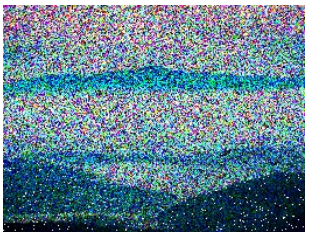
VIII. INTRODUCTION

Noise can make images look messy and unclear. In this project, we worked on two noisy images: a line drawing of Iron Man with white specks, and a scenic image with different types of noise. We tried several filtering methods—like Connected Component Filtering, Bilateral Filtering, Non-Local Means, Adaptive Filtering, and Frequency Low-Pass Filtering—to remove noise while keeping the main details. By testing and adjusting these methods, we were able to improve the quality of both images.

IX. PROBLEM STATEMENT

I was provided with two corrupted image files: `iron_man_noisy.jpg` and `scenary_noise.jpg`. The first image contains a line drawing of a subject (Iron Man), but the signal is heavily degraded by high-frequency "salt noise" (random white dots scattered across the frame). The goal was to write a script that filters out these artifacts to restore the clean line drawing as much as possible.





X. RELATED WORK

1. Connected Component Filtering

It is a technique used to remove small unwanted noise from a binary image. It works by finding groups of connected pixels called connected components. The size (area) of each component is calculated, and very small components are removed. This helps keep the important parts of the image while eliminating noise, resulting in a cleaner image for further processing.

2. Bilateral Filter

The Bilateral Filter is a non-linear filter used for image denoising while preserving edges. Unlike basic filters that only use spatial distance, it also considers differences in pixel intensity. It averages nearby pixels but keeps sharp edges in the image. This reduces noise without strongly blurring important details.

3. Non-Local Means (NLM)

The Non-Local Means algorithm removes noise by comparing similar patches across the whole image, not just nearby pixels. It calculates a weighted average of pixels with similar patterns, even if they are far apart. This method preserves textures and fine details while reducing noise.

4. Adaptive Filter

An Adaptive Filter changes its filtering based on local image properties such as mean and variance. It applies stronger smoothing in noisy areas and less smoothing near edges and detailed regions. This helps reduce noise while preserving important image details.

4. Frequency Domain Low-Pass Filter (LPF)

A Low-Pass Filter in the frequency domain first converts the image using the Fourier Transform. High-frequency components, which often represent noise and sharp changes, are reduced while low-frequency components are kept. The image is then converted back to the spatial domain to produce a denoised image.

XI. INITIAL ATTEMPTS

For the first image, I used Connected Component Filtering and adjusted the `MIN_COMPONENT_AREA` variable. Since noise appears as small isolated dots, components below a minimum area threshold were removed.

For the second image, I tried various methods and adjusted their repective parameters to obtain a clean image.

1. Median + Bilateral
2. Non-Local Means
3. Frequency Low-Pass
4. Adaptive Deviation Filter

Instead of trying them separately I made a new file to compare them and display their results in a single time. This code Runs all methods, Shows outputs, Saves all results, Displays them in one window. Now I turn by turn adjusted parameters of each type to obtain their best possible filtering and then compared the best to obtain the final choice.Below are the adjustments made to obtain best outputs.

1. Median + Bilateral : `median = cv2.medianBlur(img, 5) bilateral = cv2.bilateralFilter(median, 2, 100, 75)`
2. Non-Local Means : `nlm = cv2.fastNlMeansDenoisingColored(img, None, 25, 10, 7, 20)`
3. Adaptive Deviation Filter : `threshold = 70`
4. Frequency Low-Pass : `radius=65`

File : `NoiseFilter2_all.py`

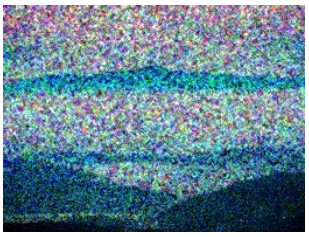
XII. FINAL APPROACH

For the first image, the final approach was more or less same as the initial approach with slight modifications which are discussed in result and observation section.

For the second image, finally the Adaptive Filter with its adjusted parameters was chosen.

XIII. RESULTS AND OBSERVATION

The final cleaned images :



Observations for Sub-task 1: Before any filtering of image, it was converted into binary format using `cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)`. This was done for the ease of noise filtering since initially the image was in grayscale. when checked with `print(np.unique(img))` it was found that it had `[ 0 1 2 3 4 5 6 7 8 9 248 249 250 251 252 253 254 255]`

Observations for Sub-task 2: For the adaptive filter, we could have a separate threshold for the three colors like `threshold =[60, 80 ,70]` (example) instead of `threshold = 70`. But this hardly made any difference because the noise was almost equally for each of these colors.

XIV. Future Work

Though the first image was almost completely cleaned but the second image was so much distorted that satisfactory cleaning could not be achieved. If possible further more advanced methods may be used to clean the noisy image of scenery.

CONCLUSION

In this project, the objective was to restore two heavily corrupted images by applying suitable noise filtering techniques. Different approaches such as Connected Component Filtering, Bilateral Filtering, Non-Local Means, Frequency Domain Low-Pass Filtering, and Adaptive Filtering were explored and compared.

For the first image containing the Iron Man line drawing, converting the image to a binary format followed by Connected Component Filtering proved effective in removing the salt noise while preserving the main structure of the drawing. By tuning the minimum component area parameter, most of the isolated noise pixels were successfully eliminated.

For the second image, multiple denoising techniques were tested and their outputs were compared by adjusting their respective parameters. Among the tested methods, the Adaptive Deviation Filter produced comparatively better results in reducing noise while retaining the overall structure of the image.

Overall, the project demonstrated that selecting an appropriate filtering technique depends strongly on the type and distribution of noise present in the image. Through experimentation and parameter tuning, it was possible to significantly improve the visual quality of the corrupted images.

Medial Axis Detection

***Abstract*—Medial Axis Detection of Moving Objects. Detection of Medial Axis in the provided three videos.**

XV. Introduction

The objective of this task was to detect the medial axis of a tool from a video sequence. The medial axis represents the centerline of the tool and is useful for estimating its orientation and position. To achieve this, I gradually built a processing pipeline while experimenting with different image processing techniques.

XVI. Problem Statement

The primary task was to process video clips of a moving surgical tool to highlight its Medial Axis (the central skeletal line) on the original frames.

XVII. Related Work

1. Background Subtraction

It is a common computer-vision technique used to separate moving foreground objects from a static background in videos or image sequences. It introduces several built-in OpenCV algorithms—BackgroundSubtractorMOG, BackgroundSubtractorMOG2, and BackgroundSubtractorGMG—which model the background over time and compute a foreground mask by comparing each frame to the learned background. Each method has different strengths (e.g., MOG2 adapts to illumination changes, GMG uses Bayesian modeling).

2. Morphological transformations

They are some simple operations based on the image shape. It is normally performed on binary images. It needs two inputs, one is our original image, second one is called structuring element or kernel which decides the nature of operation. Two basic morphological operators are Erosion and Dilation. Then its variant forms like Opening, Closing, Gradient etc also comes into play.

3.The Sobel Operator

It is an image processing technique used to detect edges in an image. It finds edges by calculating how quickly the brightness (intensity) of pixels changes in different directions. The method uses small filters called kernels to measure changes in two directions:

Horizontal direction (Gx)

Vertical direction (Gy)

These gradients are then combined to determine the strength and direction of edges in the image. Usually, the image is converted to grayscale and slightly blurred before applying the Sobel operator to reduce noise.

4.The Hough Transform

It is an image processing technique used to detect straight lines in an image. It works even if the lines are partially broken or noisy. The method represents every possible line using two parameters:

ρ (rho): distance of the line from the origin

θ (theta): angle of the line

For each edge point in the image, the algorithm calculates all possible (ρ, θ) values that could form a line through that point. These values are stored in a grid called an accumulator. Each point "votes" for the lines it could belong to. When many points vote for the same (ρ, θ) pair, that location gets a high count, indicating that a line exists in the image with those parameters. Usually, edge detection (like Canny) is applied first so that the algorithm works only on important edge pixels.

XVIII. Initial Attempts

Before arriving at the final solution, I tried several approaches.

I began by extracting the video frames and applying morphological operations to isolate the the moving portion from the background. My initial attempt involved applying erosion to reduce noise in the foreground mask obtained from background subtraction.

But later,I replaced the erosion operation with a morphological opening technique. Opening performs erosion followed by dilation. This produced a cleaner foreground.

I then experimented with combining opening and closing operations to further improve the mask in hope to get better results. However, after testing this combination on multiple frames, I observed that the foreground has lost its sharpness and clarity so, I removed the closing step and focused on optimizing the opening operation.

Next, I tested different kernel sizes for the morphological operations. After observing several video frames and trying on all the three provided videos, I found that a kernel size of (6,6) worked best.

Once the foreground mask was clean, I started the edge detection step. I added edge detection using the Sobel operator to highlight the boundaries of the tool.

After detecting edges, I implemented the Hough Transform. Initially, the algorithm detected a very large number of lines, many of which corresponded to the same edge region.

So, I added a threshold on the edge image so that only strong edges would contribute to the Hough voting process. I also reduced the angular resolution of the Hough transform to 4, which significantly reduced the number of votes and improved the speed of the algorithm.

Further, I sorted the detected lines by their vote counts and selected only the strongest few lines. This made the detection more stable.

At the end, I used them to compute the medial axis in the final approach.

XIX. Final Approach

First, I read the video frame by frame. For each frame, I applied background subtraction using the MOG2 algorithm to separate the moving object (the surgical tool) from the background.

To reduce this noise, I applied a morphological opening operation using an elliptical kernel.

After cleaning the foreground, I focused on detecting the edges of the tool. I first applied a Gaussian blur to smooth the image and reduce small variations. Then I used the Sobel

operator in both the horizontal and vertical directions to compute the image gradients.

Next, I combined the horizontal and vertical gradients to obtain a single edge image. I then applied a binary threshold to keep only strong edges and remove weaker ones.

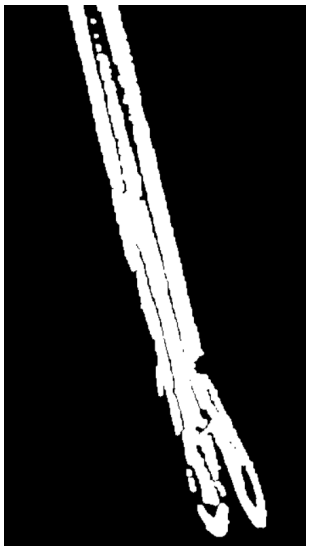
Once I obtained the edge pixels, I implemented a the Hough Transform to detect straight lines. In this method, each edge pixel votes for possible lines in the parameter space defined by rho (distance from the origin) and theta (angle of the line). I stored these votes in an accumulator array. The peaks in this accumulator represented potential lines present in the image.

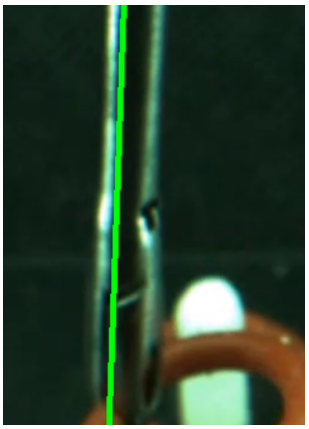
After building the accumulator, I selected the lines that received votes above a chosen threshold. I then sorted these detected lines based on their vote counts and selected the strongest few lines.

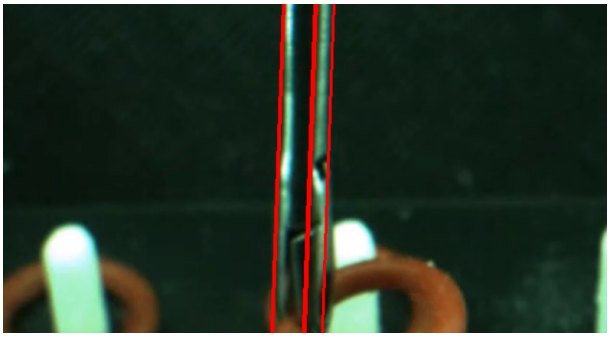
To estimate the medial axis of the tool, I used the two strongest detected lines. Since these lines correspond to the edges of the tool, I approximated the centerline by averaging their parameters (rho and theta).

I drew this averaged line on the original video frame in green, which represents the estimated medial axis of the tool.

XX. Results and Observation







Observations :

1. For exiting the running video I have used Esc key. in the code i have used `cv2.waitKey(30) & 0xff`. This keeps only the last 8 bits of the returned key code. Different systems (Windows/Linux/Mac) may return key codes with extra bits. & 0xff ensures you get the standard ASCII key value.

2. Intially when I applied MOG2 then it showed some extra objects which which in gray color rather than white. Those were the shadows of moving objects which were unwanted. So I did detectShadows=False to disable them.

3. Rather than the rectangle kernel here i have used elliptical kernel. this was done because An ellipse (elliptical) kernel is generally better than a rectangle kernel when:Objects are rounded or curved – Elliptical kernels preserve curved edges better and avoid sharp corners.

XXI. Future Work

Many different techniques might have been used which may provide better results. One possible improvement is better isolation of the tool from the background. At present, the method relies mainly on edge detection, which can sometimes capture unwanted edges from the background. Using contour detection and selecting the largest contour or applying proper segmentation could help ensure that only the tool is processed.

Conclusion

In this project, a complete pipeline was developed to detect the medial axis of a moving surgical tool in a video. First, background subtraction was used to separate the moving tool from the static background. Morphological operations were then applied to remove noise and obtain a clean silhouette of the object. After cleaning the image, edge detection was performed using the Sobel operator to identify the boundaries of the tool. A custom implementation of the Hough Transform was then used to detect the straight edges of the tool without relying on built-in functions. Finally, the two strongest detected lines were used to estimate the centerline of the tool, and their average was drawn on the original frame as the medial axis. This approach successfully highlights the central axis of the moving object while following the given constraints.

References

[1] https://www.geeksforgeeks.org/python/python-program-extract-frames-using-opencv/

[2] https://docs.opencv.org/3.3.0/db/d5c/tutorial$_py_bg_subtraction.html$?

[3] https://docs.opencv.org/3.0-beta/doc/py$_tutorials/py_imgproc/py$ _morphological_ops/py_morphological_ops.html?

[4] https://docs.opencv.org/3.4/d2/d2c/$tutorial_sobel_derivatives.html$?

[5] $https://docs.opencv.org/3.0-beta/doc/py_tutorials/py_imgproc/$ py_houghlines/py_houghlines.html?